

Chapter 10

玩家角色-移動

Horazon

手機程式設計

複習：上週重點

- [x] 主角有了 Capsule Collider 2D (不會卡牆角)。
- [x] 主角有了 Rigidbody 2D (有重力，且鎖定 Z 軸不會跌倒)。
- [x] 主角不會黏牆壁 (加上了 Slippery 零摩擦材質)。
- [x] 建立了 `PlayerController.cs` 腳本空殼。

今天，我們要賦予他**行動力**，讓他可以左右跑動！

本章目標

1. 理解 Unity Input Manager (輸入管理系統)。
2. 讀取鍵盤與搖桿訊號 (`Input.GetAxisRaw`)。
3. 使用物理速度 (`Rigidbody.velocity`) 來移動角色。
4. 解決「太空漫步」問題 (透過縮放來實現角色翻面)。

輸入系統：Input Manager

Unity 非常貼心，幫我們把鍵盤、滑鼠、甚至遊戲搖桿都整合好了。

1. 從上方選單點選：Edit -> Project Settings -> 找到 Input Manager。
2. 展開 Axes -> 展開第一個 Horizontal (水平輸入)。
3. 你會看到裡面的設定：
 - Negative Button: `left` (鍵盤左鍵), `a` (A鍵)。
 - Positive Button: `right` (鍵盤右鍵), `d` (D鍵)。
 - Gravity / Sensitivity: 按鍵靈敏度設定。

💡 **防呆提示：**我們寫程式時，只要呼叫 `"Horizontal"` 這串字，Unity 就會自動幫我們偵測這些對應的按鍵，不用自己寫又臭又長的判斷式！

讀取輸入 (Coding)

在我們上週建立的 `PlayerController.cs` 中，加入 `Update` 區塊：

```
float mx; // 宣告一個變數存放左右輸入的數值

void Update()
{
    // 讀取水平輸入 (數值範圍：-1 ~ 1)
    // 按左鍵會變成 -1，按右鍵會變成 1，不按是 0
    mx = Input.GetAxisRaw("Horizontal");
}
```

- `GetAxis`：有加減速緩衝，放開按鍵會慢慢滑行停止 (適合賽車遊戲)。
- `GetAxisRaw`：反應非常直接，0 瞬間變 1 或 -1，適合 2D 動作遊戲，操作手感更靈敏。

施加移動 (Velocity)

有了輸入訊號 `mx`，我們要推動角色的物理剛體 (`Rigidbody 2D`)。

💡 防呆提示：請一律在 `FixedUpdate` 中執行物理指令！

```
void FixedUpdate()  
{  
    // 設定剛體的速度 (Velocity)  
    // X軸 (左右) = 輸入數值 (mx) * 移動速度 (moveSpeed)  
    // Y軸 (上下) = 維持原本的速度 (rb.velocity.y)  
    rb.velocity = new Vector2(mx * moveSpeed, rb.velocity.y);  
}
```

[!WARNING]

千萬不要寫 `new Vector2(mx * moveSpeed, 0)`，把 Y 軸設為 0 會抵銷重力，角色在空中會掉不下來，變成騰雲駕霧！

完整程式碼預覽

確認你的 `PlayerController.cs` 長得像這樣：

```
using UnityEngine;

public class PlayerController : MonoBehaviour
{
    public float moveSpeed = 5f; // 可以在 Inspector 調整速度
    public Rigidbody2D rb;
    float mx; // 存放輸入數值

    void Start() {
        rb = GetComponent<Rigidbody2D>(); // 自動抓取剛體
    }

    void Update() {
        // 每幀讀取玩家輸入
        mx = Input.GetAxisRaw("Horizontal");
    }

    void FixedUpdate() {
        // 在固定幀率下執行物理移動
        rb.velocity = new Vector2(mx * moveSpeed, rb.velocity.y);
    }
}
```

實作測試 1：移動看看！

1. 存檔 (Ctrl+S)，回到 Unity。
2. 按下上方的 **Play** 按鈕。
3. 按下鍵盤的 **A / D** 或 **左右方向鍵**。
4. 主角應該會左右平滑移動了！
5. **調整手感：**
 - 在右側 **Inspector** 找到腳本裡的 **Move Speed**。
 - 試試看輸入 5, 8, 10，找到你覺得最順手的跑步速度。

問題發現：太空漫步 (Moonwalking)

當你往左走的時候，你會發現主角的臉...還是朝右邊！

這看起來就像麥可傑克森在跳月球漫步。

為了解決這個問題，我們需要在往左走時，把角色的圖片「翻過來」。

翻面邏輯 (Flipping)

在 Unity 中翻轉角色有兩種主流方法：

1. ❌ `SpriteRenderer.flipX`：

- 只翻轉主角那張圖片。
- **缺點**：如果你主角手上有拿槍 (子物件)，圖片翻過去了，但槍不會跟著翻過去，會變成背對著開槍。

2. ✅ `Transform.localScale`：

- 把 Transform 的 X 軸縮放比例改為 `-1`。
- **優點**：所有掛在主角底下的子物件 (槍、背包、檢測器) 都會一起完美翻轉。
- **這才是專業的做法，我們採用這個！**

撰寫翻轉程式碼

回到 `PlayerController.cs`，在 `Update()` 裡面加入判斷邏輯：

```
void Update()
{
    mx = Input.GetAxisRaw("Horizontal");

    // 如果輸入向右 (大於 0)
    if (mx > 0)
    {
        // 縮放設為正數，面朝右
        transform.localScale = new Vector3(1, 1, 1);
    }
    // 如果輸入向左 (小於 0)
    else if (mx < 0)
    {
        // x軸縮放設為負數，面朝左
        transform.localScale = new Vector3(-1, 1, 1);
    }
    // 💡 防呆提示：如果 mx == 0 (沒按鍵)，甚麼都不做，保持原本的面朝方向
}
```

實作測試 2：完美翻轉

1. 存檔，回到 Unity。
2. 按下 Play。
3. 往左走 -> 角色完美變身為左撇子，面朝左邊。
4. 往右走 -> 角色變回來面朝右邊。
5. 💡 **觀察技巧**：一邊走一邊看 `Inspector` 的 `Transform -> Scale x`，數值會在 1 和 -1 之間自動切換。

進階技巧：跑步功能 (Sprint)

大部分的遊戲都有「按住 Shift 鍵可以加速跑」的功能。
我們可以輕鬆加上這個機制！

1. 在最上方多宣告一個變數：`public float runSpeed = 8f;`
2. 在 `Update()` 中加入按鍵判斷：

```
float currentSpeed; // 決定當下的速度

if (Input.GetKey(KeyCode.LeftShift)) {
    currentSpeed = runSpeed; // 按住 Shift，變成跑步速度
} else {
    currentSpeed = moveSpeed; // 沒按，維持走路速度
}
```

3. 在 `FixedUpdate()` 裡，把原本的 `moveSpeed` 替換成 `currentSpeed`。

跨平台支援 (Gamepad)

你知道嗎？你剛剛寫的這段簡單的程式碼，**已經自動支援搖桿了！**

- `Input.GetAxis("Horizontal")` 預設就會去抓 Xbox 或 PS 手把的**左類比搖桿**。
- 如果你手邊有手把，插上電腦，直接就能操作主角！
- 這就是 Unity `Input Manager` 系統強大的地方，幫我們省下了處理硬體對接的麻煩。

常見問題排解 (Debug)

Q: 角色移動感覺很黏、很慢？

A:

1. 檢查 `moveSpeed` 數值是不是太低。
2. 檢查剛體的 `Linear Drag` (線性阻力)，在空氣中移動通常設為 0，除非你在做水下關卡。

Q: 角色放開按鍵後，還會往前滑行一段距離？

A:

1. 確認你使用的是 `GetAxisRaw` 而不是 `GetAxis`。
2. 如果你使用了 `AddForce` (施力模式) 會有慣性，但我們這裡是直接設定 `velocity`，理應會瞬間停止。

總結

今天我們完成了角色控制的三大核心：

1. **讀取玩家輸入**：使用 `Input.GetAxisRaw("Horizontal")`。
2. **物理移動執行**：在 `FixedUpdate` 中設定 `rb.velocity`。
3. **角色左右翻轉**：判斷輸入方向，並修改 `transform.localScale`。

主角終於聽話了！現在可以在關卡裡自由自在地左右奔跑。

下週預告

身為一個動作遊戲主角，只能左右跑絕對不夠酷。
平台跳躍遊戲的靈魂在於——**跳躍 (Jump)**。

下週我們要挑戰最容易出 Bug 的關卡：

- 如何施加準確的跳躍力道。
- **地面檢測 (Ground Check)**：主角怎麼知道自己踩到地板了？(防止在空中無限二段跳飛上天)。

(開放提問)

- 用這個寫法可以做俯視角 (Top-Down) 的 RPG 遊戲嗎？
 - 原理完全一樣！只需要多讀取一個 Y 軸輸入 (`Input.GetAxisRaw("Vertical")`)，然後把 Rigidbody 的重力設為 0，把 Y 軸輸入放進 `velocity.y` 裡就可以了。
- 為什麼移動不用 `transform.Translate` 寫法？
 - 因為 `Translate` 會直接改變座標，**無視物理碰撞體**，這會導致主角直接穿破牆壁！

(助教巡堂協助檢查程式碼)